



6synchronization-intro

Senkronizasyona Giriş

Yarış Durumu

Birden fazla işlem veya iş parçacığı (thread) aynı anda ortak bir değişkeni değiştirmeye çalışırsa **race condition** (yarış durumu) oluşur.

Kritik Bölge

Kodumuzun içinde, işte tam da bu "ortak paylaşılan verinin" (örneğin ortak bir değişkenin veya donanımın) değiştirildiği o tehlikeli ve hassas bölüme **Kritik Bölge (Critical Section)** adını veriyoruz.

Temel amacımız şu: Bir işlem bu bölgeye girdiyse ve veriyi güncelliyorsa, başka hiçbir işlem aynı anda o bölgeye giremesin.

Kritik bölge problemini çözmek için geliştireceğimiz herhangi bir mantığın şu üç şartı **kesinlikle** sağlaması gerekir:

1. **Karşılıklı Dışlama (Mutual Exclusion):** Tek şeritli dar bir köprü düşün. Eğer A işlemi o an kritik bölgedeyse (köprüden geçiyorsa), B işlemi kesinlikle o bölgeye giremez.
2. **İlerleme (Progress):** Diyelim ki köprü boş (kritik bölgede kimse yok) ve köprüden geçmek isteyen işlemler var. Sistemin sonsuza kadar "Kim geçecek?" diye kararsız kalmaması, işlemlerden birini seçip trafiği bir an önce akıtması gerekir.
3. **Sınırlı Bekleme (Bounded Waiting):** Köprüden geçmek için bekleyen bir araba düşün. Diğer arabalar sürekli onun önüne geçip kaynakları tüketmemelidir. Bir işlem sıraya girdikten sonra, onun önüne geçebilecek diğer işlemlerin sayısı sınırlı olmalıdır ki sonsuza kadar aç (starvation) kalmasın.

İlkel Çözüm: Kesmeleri (Interrupts) Kapatmak

Bir işlem kritik bölgeye girerken, işletim sistemine "Beni bölme, dış dünyayı tamamen kapat!" der. İşini bitirince tekrar kesmeleri açar.

İşlem bir döngüde sıkışıp bütün bilgisayarı dondurabilir. Ayrıca, günümüzdeki multi-core işlemcilerde sadece 1 çekirdeği kapatacağı için işlem çalışmaz.

Yarış Durumunu Çözmek için Çözümler

Mutex (Mutual Exclusion)

1. Mutex Nedir?

Mutex, kelime anlamı olarak "Karşılıklı Dışlama" demektir ve Kritik Bölge probleminin başrol oyuncusudur.

PDF'teki o harika benzetmeyle anlatalım: Mutex'i kalabalık bir toplantıdaki **"tek bir mikrofon"** olarak düşün:

- Ortak bir hafıza alanında işlem yapma (konuşma) hakkı, sadece mikrofonu elinde tutan iş parçacığındadır (thread).
- Eğer sen konuşmak istiyorsan ve mikrofon başkasındaysa, o kişi konuşmasını bitirip mikrofonu masaya bırakana (unlock) kadar beklemek zorundasın.
- **Altın Kural:** Mikrofonu (kilidi) kim aldıysa, işi bitince masaya bırakacak olan da odur. Başka bir thread senin kilidini açamaz.

2. Kilidi Nerede Kullanmalıyız?

Diyelim ki devasa bir dizideki elemanları toplayacağız. Eğer her bir sayıyı toplarken "mikrofonu al - sayıyı ekle - mikrofonu bırak" yaparsan, yani bunu 1 milyon kez tekrarlarsan, programın inanılmaz yavaş çalışır.

Doğru mantık şudur: Her iş parçacığı kendi kendi yerel değişkeninde 1 milyon sayıyı hızlıca toplasın. İşlem bitince, sadece global değişkene yazacağı o tek an için kilidi alsın ve işini bitirsin. Kilit işlemleri maliyetlidir, olabildiğince az ve öz kullanılmalıdır.

Condition Variables (Durum Değişkenleri)

Mutex harikadır ama sadece "erişimi" kontrol eder. Bazen verinin erişiminden ziyade, **verinin güncel değerine** göre senkronize olmamız gerekir.

Örneğin; bir thread, bir listeye eleman ekleyecek ama liste dolu. Ne yapacak? Sürekli "Liste boşaldı mı?" diye CPU'yu yorması mantıksızdır.

İşte burada **Durum Değişkenleri (Condition Variables)** devreye girer. Bu değişkenler sayesinde thread kilidi geçici olarak bırakır ve "Şartlar değiştiğinde beni uyandırın" diyerek uykuya dalar. Başka bir thread listeden eleman çıkarıp yer açtığında, uyuyan thread'e haber verir ve uyandırır.



Durum değişkenleri **her zaman** bir Mutex ile omuz omuza çalışır.

Wait (Bekleme) Fonksiyonunu

Bir iş parçacığı (thread), aradığı şart sağlanmadığında uykuya dalmak için `wait` çağırısı yapar.

Uyuma işlemini kodlarken hiçbir zaman `if(şartlar uygun değilse)` kullanmayız, her zaman `while (şartlar uygun değilse)` şeklinde bir döngü kurarız.

Peki ama neden? Çünkü sen uyandırıldığında, kilidi tekrar eline alana kadar geçen o milisaniyelik sürede, başka bir thread araya girip senin beklediğin kaynağı tüketmiş olabilir. Uyandığında "Evet bana uyan dendi ama şartlar hala benim için uygun mu?" diye tekrar kontrol etmen gerekir. Bu "sahte uyanmalar" (spurious wakeups) veya "araya girmeler" ihtimaline karşı while döngüsü senin sigortandır.

Monitors (Monitörler)

Monitör, aslında üstte anlatılan şeylerin paketlenmiş, güvenli bir "Tasarım Şablonu"dur.

İnsan hatasını ortadan kaldırmak için tasarlanmış **yüksek seviyeli (high-level) bir senkronizasyon yapısıdır**.

Bir monitörün içinde şunlar bulunur:

- Veriyi korumak için mutex,

- Thread'lerin beklemesini ve uyanmasını koordine eden **en az bir adet Durum Değişkeni (Condition Variable)**.

Monitör, paylaşılan verileri (değişkenleri) ve bu veriler üzerinde işlem yapacak fonksiyonları içine hapseden bir kapsüldür.

- **Otomatik Kilit (Mutual Exclusion):** Bir process (işlem) veya thread monitörün içindeki bir fonksiyona girdiği anda, monitörün kapısı diğer herkese kilitlenir. İçerideki işlem bitmeden başka hiçbir işlem monitöre giremez. Dışarıdakiler "Giriş Kuyruğunda" (Entry Queue) bekletilir.

Koşul Değişkenleri (Condition Variables) ile Deadlock Çözümü

Diyelim ki bir Thread, Monitör'ün içine girmeyi başardı. Kapı da arkasından kilitlendi. Fakat içeride işini yaparken bir baktı ki çalışması için gereken bir koşul henüz sağlanmamış (örneğin; okuyacağı dosya henüz diske gelmemiş veya Buffer tamamen boş).

- **Kriz:** Eğer bu Thread *"Dosya gelene kadar burada uyuyayım"* derse, Monitör kilitli kaldığı için dosyayı getirecek olan diğer Thread asla içeri giremez. Sistem kilitlenir.
- **Çözüm:** Monitörler bu krizi **Koşul Değişkenleri (Condition Variables)** ile çözer (Genelde `x`, `y` gibi isimlendirilirler).

Bir koşul değişkeni üzerinde sadece iki işlem yapılabilir:

- `x.wait()` : İçerideki Thread bu komutu çağırdığında, **Monitörün kilidini bırakır** ve o koşulun (x'in) kendi özel bekleme kuyruğunda derin bir uykuya dalar. Böylece kapı açılır ve diğer Thread'ler içeri girebilir.
- `x.signal()` : İçeri giren başka bir Thread işini halledip (örneğin Buffer'a veriyi yazıp) bu komutu çağırdığında, `x` kuyruğunda uyuyan **sadece bir tane** Thread'i uyandırır. (Eğer kuyrukta kimse yoksa `signal` havaya gider, hiçbir etkisi olmaz. Semaforlardaki gibi değeri artırmaz!).

Semaforlar (Semaphores)

Mutex'i "tek bir mikrofon" olarak tanımlamıştık. Peki ya içeride 10 tane mikrofon veya 10 tane boş sandalye varsa? İşte **Semafor**, içinde bir sayaç barındıran senkronizasyon aracıdır.

- Diyelim ki semaforumuzu 10 değeriyle başlattık (10 boş sandalye var).
- **Wait (Azaltma / Down / P):** Bir thread gelip "bana bir kaynak ver" dediğinde sayacı 1 azaltır ve sandalyeye oturur. Eğer sayaç zaten 0 ise (boş sandalye yoksa), sayı 0'dan büyük olana kadar uyuyarak bekler.
- **Post / Signal (Artırma / Up / V):** İşini bitiren thread kalkar, "ben sandalyemi boşalttım" der ve sayacı 1 artırır. Bu sayede bekleyenlerden biri uyanıp oturabilir.
 - Arttırma işleminin sınırı yoktur, 10 boş sandalyeyle başlanmış olsa da +10 müşteri kalkarsa sayaç da artmaya devam eder.

Mutex ve Binary Semaphore Farkı

Eğer bir semaforun maksimum sayısını 1 yaparsak (sadece 0 ve 1 değerini alabiliyorsa), buna **Binary (İkili) Semafor** denir.



Bu semafor diğerleri gibi istediği kadar artabilir değildir, sadece 0 ve 1 değerlerini alır.

Görünüşte tıpkı Mutex gibi sadece tek bir kişinin girmesine izin verir. Ama aralarında dağlar kadar büyük, felsefi bir fark vardır:

- Eğer amacın "**Bu değışkene benden başka kimse dokunmasın**" demekse, Mutex kullanırsın; kilidi sen alır, sen bırakırsın.
- Eğer amacın "**Ben işimi bitirmeden sen başlama, bekle! Ben bitirince sana uzaktan sinyal gönderip kapını açacağım**" demekse, Semafor kullanırsın; çünkü Semafor'da `sem_wait` ve `sem_post` komutlarını bambaşka thread'ler çağırabilir.

Deadlock & Livelock

- **Deadlock (Kördüğüm):** İki veya daha fazla iş parçacığının, birbirlerinin elindeki kaynakları sonsuza kadar beklemesi durumudur.
- **Livelock (Aktif Kilitlenme):** Bir iş parçacığının, diğer iş parçacığının eylemine yanıt olarak sürekli durum değiştirmesidir. Dar bir koridorda karşılıklı yürüyen

iki kiřinin birbirine yol vermek için aynı anda saęa ve sola adım atıp sürekli birbirlerinin yolunu kesmesi (aktif olarak hareket etmeleri ama hiç ilerleme olmaması) gibi düşünebilirsiniz.